KATALYZE: AN AUTONOMOUS TESTING AGENT FOR FUNCTIONAL BUGS IN WEB APPLICATIONS

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

by

ARUNODAY A K (CB.SC.U4AIE23106)
ACHUTHARAJ R (CB.SC.U4AIE23108)
LOKESH D (CB.SC.U4AIE23123)
ABHAY ROHIT M (CB.SC.U4AIE23173)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**Katalyze: An Autonomous Testing Agent for Functional Bugs in Web Applications**" submitted by **Arunoday A K, Roll. No: CB.SC.U4AIE23106**, **Achutharaj R, Roll. No: CB.SC.U4AIE23108**, **Lokesh D, Roll. No: CB.SC.U4AIE23123** and **Abhay Rohit M, Roll. No: CB.SC.U4AIE23173** for the award of the **Degree of Bachelor of Technology** under the **Faculty of Engineering** is a bonafide record of the work carried out by them under my guidance and supervision at "Department of Artificial Intelligence", Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on .....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Palmani
Thesis Advisor,
Assistant Professor (Sr. Gd.),
Center for Computational Engineering and Networking,
School of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS

DECLARATION

We, **Arunoday A K**, **Reg. No: CB.SC.U4AIE23106**, **Achutharaj R**, **Reg. No: CB.SC.U4AIE23108**, **Lokesh D**, **Reg. No: CB.SC.U4AIE23123** and **Abhay Rohit M**, **Reg. No: CB.SC.U4AIE23173** hereby declare that this thesis entitled "**Katalyze: An Autonomous Testing Agent for Functional Bugs in Web Applications**" is the record of the original work done by us under the guidance of **Dr. Palmani**, Assistant Professor (Sr. Gd.), Center for Computational Engineering and Networking, School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree/diploma/associateship/fellowship or a similar award to any candidate in any University.

Place: **Signature of the Students**

Date:

COUNTERSIGNED

Dr. K.P Soman
Professor and Dean,
Department of Artificial Intelligence
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

Contents

Acknowledgements **iv**

List of figures **v**

List of tables **vii**

Abbreviations **viii**

List of symbols **ix**

Abstract **x**

1 Introduction **1**

1.1 Motivation 1

1.2 Problem statement 1

1.3 What a functional bug is 2

1.4 Objectives 3

1.5 Contributions 4

1.6 Scope 5

1.7 Organisation of the thesis 5

2 Background and Related Work **6**

2.1 Automated web testing and test oracles 6

2.2 Reinforcement learning for web and GUI testing 7

2.3 Language models in testing and as reward 8

2.4 Deep reinforcement learning and exploration 8

2.5 Evaluating testing tools 9

2.6 Research gap 10

3 System Overview and the Verified-Fault Oracle **11**

3.1 The browser environment 11

3.1.1 Action registry 12

3.1.2 Deterministic triggers 12

3.2 The verified-fault oracle 12

3.2.1 Seeded faults and assertions . . . 12
3.2.2 From candidate to verified finding . . . 13
3.2.3 Crash-type faults . . . 13
3.2.4 Specification-free invariants . . . 14
3.3 The product layer . . . 14

4 A Deep Reinforcement Learning Tester (v1) 15
4.1 Testing as a Markov decision process . . . 15
4.2 The agent: an action-conditioned dueling double DQN . . . 16
4.3 Reward . . . 16
4.4 Experiment 1: does pretraining transfer? . . . 17
4.5 Experiment 2: context or memorisation? . . . 19
4.6 Checks of the LLM components . . . 19
4.7 What v1 taught . . . 20

5 The Hybrid Tester (v2) 21
5.1 Page abstraction . . . 21
5.2 Archive with exact return . . . 22
5.3 Test operators . . . 22
5.4 The decision layer . . . 23
5.5 Options for the final tuning round . . . 24
5.6 Development rounds . . . 24

6 Test Applications and Evaluation Methodology 27
6.1 The six-application benchmark . . . 27
6.2 The generated application family . . . 27
6.3 The development suite . . . 27
6.4 The validation suite . . . 28
6.5 The external benchmark . . . 29
6.6 Admission . . . 29
6.7 Data split and independence . . . 30
6.8 Metrics and statistics . . . 30
6.9 Threats to validity . . . 31

7 Evaluation 32
7.1 Held-out evaluation (pre-registered) . . . 32
7.1.1 Design . . . 32
7.1.2 Results . . . 33
7.1.3 What the held-out data shows (descriptive) . . . 34
7.2 Tuning round v3 on the rebuilt development suite . . . 34
7.2.1 Design . . . 34
7.2.2 Results . . . 35

7.2.3 Configuration chosen . . . 36
7.3 Comparison with WebExplor and WebRLED . . . 36
7.3.1 Design . . . 36
7.3.2 Results . . . 37
7.4 Post-hoc follow-up: the action space . . . 39
7.5 Discussion . . . 40

8 Conclusion and Scope for Further Research **43**
8.1 Conclusion . . . 43
8.2 Limitations . . . 44
8.3 Scope for further research . . . 44

A Agent Configurations **45**
A.1 Arms . . . 45
A.2 Held-out evaluation per fault . . . 46

References **47**

Acknowledgements

We thank our thesis advisor, Dr. Palmani, for the guidance given throughout this project, for questioning our results whenever they looked too good, and for pushing us to state our limitations openly. We thank Dr. K.P. Soman, Professor and Dean, and the faculty of the Department of Artificial Intelligence for the facilities and the review process that shaped this work. We are grateful to the researchers who made their web-testing subject applications public: the authors of WebExplor and WebRLED, and Matteo Biagiola and colleagues, whose replication material packages those applications with their instrumentation. Without open benchmarks, an honest comparison with published work would not be possible. Finally, we thank our families and friends for their support.

ARUNODAY A K
ACHUTHARAJ R
LOKESH D
ABHAY ROHIT M

List of Figures

3.1 The system around the explorer. Only the explorer differs between v1 and v2. . 11

5.1 The hybrid tester's loop. Every replayed action counts towards the budget. . . . 21

List of Tables

1.1 The classes of functional bug used in this thesis, with an example from the test applications and the oracle check that observes the failure. 3

4.1 Experiment 1: discovery per arm over all folds (runs of 2,000 interactions). . . . 18
4.2 Experiment 1: pre-registered comparisons (negative Δ = faster). 18
4.3 Experiment 2: pre-registered verdicts for the full agent. 19
4.4 LLM labeller accuracy against ground truth (gates 0.85 / 0.80). 20
4.5 Findings from v1 and the design consequences for v2. 20

5.1 Development rounds of the hybrid tester and their decisions. 25
5.2 All arms on the same runs: suite rounds 2–5, 14 applications $\times$ 3 seeds. Difference is the per-application mean T against `hybrid_nolearn`, with a bootstrap interval over applications (negative = faster). 25

6.1 The development suite (version 2). 28
6.2 The validation suite, used once, for the held-out evaluation. 29

7.1 Held-out evaluation: pre-registered comparisons (negative Δ = first arm faster). 33
7.2 Held-out evaluation per arm (40 runs each). 33
7.3 Held-out evaluation per application: runs that found a fault (of 5) and mean T. 33
7.4 Tuning round v3 per arm: 24 runs each on the eight rebuilt development applications, which hold 41 seeded faults (8 of them crash-type). 35
7.5 Tuning round v3 against `hybrid_nolearn`: per-application mean difference in T (negative = faster), median over applications with a 95% bootstrap interval. Apps with no difference are not counted as faster or slower. 35
7.6 External comparison: client-side branch coverage (%), mean [min, max] over three one-hour runs, beside the published means (five runs) of WebExplor and WebRLED [1]. 38
7.7 Post-hoc: distinct console error messages per run (mean [min, max]), the measure closest to the papers' manually deduplicated failures. 38
7.8 Post-hoc follow-up: branch coverage (%, mean of three one-hour runs) without and with custom clickables, all on the same hardware. 39
7.9 The best arm per measure, across all evaluations. 41

A.1 Every arm used in the evaluations, by generation. 45

A.2 Hyperparameters. 46
A.3 Held-out evaluation: runs (of 5) that verified each fault. 46

Abbreviations

AC-DQN	–	Action-Conditioned Deep Q-Network
AUROC	–	Area Under the Receiver Operating Characteristic curve
CRUD	–	Create, Read, Update, Delete
DOM	–	Document Object Model
DQN	–	Deep Q-Network
GUI	–	Graphical User Interface
LLM	–	Large Language Model
MDP	–	Markov Decision Process
PER	–	Prioritised Experience Replay
RL	–	Reinforcement Learning
RMST	–	Restricted Mean Survival Time
SPA	–	Single-Page Application
UI	–	User Interface

List of symbols

s, s'	–	State before and after an action
a	–	Action (one browser interaction)
$\phi(a)$	–	Feature vector describing action a
$Q(s, \phi(a))$	–	Action-conditioned action value
γ	–	Discount factor
ϵ	–	Exploration rate
$\Phi(s)$	–	LLM-assessed task progress of state s, in $[0, 1]$
λ	–	Weight of the LLM reward term
B	–	Interaction budget of one run (2,000)
T	–	Interactions to the first verified fault ($T = B$ if none)
$\mathrm{Beta}(\alpha, \beta)$	–	Beta posterior of a bandit key
$N(s, a, s')$	–	Visit count of a page transition

Abstract

Web applications fail in ways that are easy to describe and hard to find: a total that ignores a changed quantity, a validation rule that holds on one path and not on another, a value that is lost on reload. Automated exploratory testers can reach such states, but they face two problems. They must decide where to go in a large, stateful interface where the decisive actions are many steps deep, and they must prove what they find, or their reports are noise. This thesis builds and evaluates *Katalyze*, an automated functional tester for web applications that uses reinforcement learning (RL). It confirms a fault only when the fault reproduces twice in fresh browser sessions, is absent from a control build without that fault, and survives delta-debugging minimisation.

The work has two generations. The first (v1) is an action-conditioned dueling double deep Q-network that scores every available browser action, with prioritised multi-step replay, a residual head for online adaptation, and a reward shaped by a local large language model (LLM) that judges task progress. Two pre-registered experiments tested it. Pretraining on five applications did not speed up fault discovery on a sixth (median difference 0 interactions, 95% interval $[-749, +290]$). A diagnostic over twelve generated application variants found that the network does not learn which action helps in which state (within-label AUROC 0.51). Post-hoc checks showed that the LLM's progress score and its page labels did not generalise beyond the data they were tuned on.

These findings shaped the second generation (v2), a hybrid tester. It abstracts pages explicitly, keeps an archive of reached states that it returns to by exact replay (Go-Explore), and applies test operators from standard test design. A light online learner (a Thompson-sampling bandit, or count-based curiosity Q-learning) decides which operator to try. On eight held-out applications, in a pre-registered evaluation of 160 runs with 32 clean-build controls, the hybrid arms verified the most distinct faults (19 and 18 of 24, against 13 for random exploration) with zero false positives. All four pre-registered comparisons of time to first fault were inconclusive. After re-tuning on rebuilt full-stack development applications, the frozen tester was compared with WebExplor and WebRLED on their six subject applications, measured as they were measured: one-hour runs, Istanbul branch coverage and console failures. It did not reach their published numbers. It averaged 44.3% branch coverage against 55.5% and 64.2%, and 1.8 distinct console failures against 3.1 and 4.1. A post-hoc follow-up traced much of the gap to the tester's action space rather than its policy. With clickable elements that plain HTML scanning misses, coverage on Phoenix rose from 49% to 83% for every arm, level with WebRLED. The

evidence supports a modest conclusion: the structure of a tester, more than the learning in it, determines which functional faults it verifies.

Keywords: automated web testing, reinforcement learning, deep Q-network, Go-Explore, Thompson sampling, test oracle, seeded faults, survival analysis.

Chapter 1

Introduction

1.1 Motivation

Web applications carry much of everyday business: shops, banking, booking, support desks, internal tools. Their functional faults are rarely crashes. More often a page shows the wrong total after a quantity changes, accepts an input on one path that it rejects on another, or loses a value when the page is reloaded. Such faults sit behind sequences of interactions: a user must sign in, fill a form, reach a review step, go back and change something. A tester has to perform the right sequence and then notice that the result is wrong.

Writing scripted end-to-end tests for every such sequence is expensive, and the scripts cover only what their authors thought of. Automated exploratory testing instead drives a real browser through the application and searches for misbehaviour. The simplest explorer clicks at random. More capable explorers learn where to go, and reinforcement learning (RL) is a natural fit: the tester is an agent, the application is its environment, and reaching new or suspicious behaviour can be rewarded. Several RL web testers have been published, including WebExplor [2], WebQT and WebRLED [1].

Two problems limit these testers. The first is *where to go*. The decisive actions are often many steps into a workflow, the set of available actions changes on every page, and a reward based on novelty pays for reaching new pages, not for testing them. The second is *what counts as a fault*. Published RL web testers report errors in the browser console as failures [1, 2]. That is an implicit oracle [3]: it cannot see a page that renders the wrong content without an error, and it cannot say whether a reported error is reproducible or matters.

1.2 Problem statement

This thesis asks whether reinforcement learning can make an automated functional tester for web applications find *verified* faults sooner, and under what design. A fault is verified when it reproduces twice in fresh browser sessions, is absent from a control build that lacks that one fault, and survives delta-debugging minimisation [4]. The main measure is the number of browser interactions until the first verified fault, with runs that find nothing treated as

right-censored at the budget.

1.3 What a functional bug is

Fault and failure. This thesis follows the standard distinction. A *fault* is a defect in the code. A *failure* is what a user can observe when the faulty code runs: the application's behaviour departs from what it should do [3]. Testers observe failures; the faults behind them are what developers fix. The test applications here contain known, seeded faults, and the tester's job is to make each one produce an observable failure and prove it.

Definition 1.3.1 (Functional bug)**.** A *functional bug* in a web application is a fault whose failure is an observable deviation of the application's behaviour from its functional requirements. It must be:

1. **triggered** by a finite sequence of user interactions (clicks, typed values, selections, navigation) from a defined starting state;
2. **observable** through what the user can see or what the application keeps: page content, the page reached, data shown later, data that persists, or an error the browser reports;
3. **wrong** against a stated requirement: what the application's own contract says it must compute, accept, refuse, store or show.

In the terms of ISO/IEC 25010 [5], a functional bug is a failure of *functional correctness*: the product does not provide the correct results. It is not a failure of the non-functional characteristics, so the following are *not* functional bugs here:

- security vulnerabilities (out of scope, Section 1.6);
- performance;
- visual layout;
- usability;
- accessibility.

A page that renders slowly or looks wrong but computes, stores and shows the right things has no functional bug.

A taxonomy. Every seeded fault in this thesis belongs to one of six classes (Table 1.1). Each class corresponds to the assertions the oracle uses to decide that a failure occurred (Section 3.2), so the definition is operational: for every class, the thesis can state exactly what observation shows the bug.

Table 1.1: The classes of functional bug used in this thesis, with an example from the test applications and the oracle check that observes the failure.

Class	Definition	Example	Oracle check
Calculation error	A number the application shows differs from the correct computation over the user's inputs.	A basket total ignores a changed quantity.	`numeric_equals`, region arithmetic
Constraint bypass	An input the application must reject is accepted, or a state guarded by a precondition is reached without it.	An order is placed with an empty delivery address.	`reached_with_invalid_input`, `region_empty`
State corruption	Data shown later differs from what the user entered or what the application should have derived.	Express delivery chosen; the confirmation says standard.	`echoes_input`, `text_equals`
Persistence failure	Data the user saved is lost, reverted or not stored.	An edited record reverts after a reload.	`echoes_input` across pages, region stable across reload
Broken transition	An action leads to the wrong page or state, or to none.	"Next" returns to the first step of a wizard.	`url_matches`, `text_contains`
Crash-type	An interaction makes the application's own code throw an uncaught error or a request to the application fail.	Ticking a checklist item throws a script error.	`console_or_request_error`

Why the definition matters for the evaluation. The first five classes are *silent*: the page renders, nothing errors, and the result is simply wrong. An implicit oracle, one that counts console errors as published RL web testers do [1, 2], cannot see them. Only the last class is visible to such an oracle. This is why the thesis measures *verified* functional bugs against a specification of each seeded fault. It is also why the external comparison, which uses the literature's measures of coverage and console failures, sees only part of what a functional tester is for.

1.4 Objectives

1. Build a browser environment and a deterministic, replay-verified oracle for seeded faults that makes false positives measurable and rare.

2. Build a deep RL tester and test, under pre-registration, whether pretraining on other applications transfers to a new one, and whether the network learns which action helps in which state.

3. Measure whether a small, local large language model (LLM) is a useful source of reward or page labels for testing.

4. Use those findings to design a tester whose exploration does not depend on what the deep network failed to learn, and evaluate it on held-out applications under pre-registration.

5. Compare the final tester with published RL web testers on their own subject applications.

6. Package the tester as a usable product: a command-line tool and a web interface that test a URL, a static site, a Dockerfile repository or a bundled application and produce a report with evidence.

1.5 Contributions

The tester built in this thesis, product and research system together, is called *Katalyze*. Its contributions are:

1. **A verified-fault oracle for web testing.** Each seeded fault has its own control build. A candidate counts only if it reproduces twice, does not occur on that control, and survives minimisation. The oracle produced zero false positives in every clean-build control run of every evaluation.

2. **A pre-registered transfer experiment for deep RL web testing**, using leave-one-application-out evaluation and censoring-aware statistics. Its result is negative: pretraining gave no measurable benefit.

3. **A pre-registered diagnostic separating contextual learning from memorisation.** It uses a generated application family and probe states labelled by execution, and shows that the v1 network does not learn context.

4. **Two negative results for small local LLMs in testing**, each measured against ground truth: as a progress reward, and as a page and control labeller.

5. **A hybrid tester** combining page abstraction, an archive with exact return, test operators from standard test design, and a light online learner. It was evaluated on eight held-out applications under pre-registration, and verified the most distinct faults with zero false positives.

6. **Sixteen test applications with real backends and seeded faults**, each admitted by mechanical checks. The eight development applications run on PostgreSQL or MongoDB, with sign-in and crash-type faults.

7. **An external comparison with WebExplor and WebRLED** on their own applications and measures. It is a negative result, with a diagnosis: the set of actionable elements, not the policy, limited coverage.

1.6 Scope

The project began with two agents: one for security testing and one for functional testing. The security agent was dropped early so that one agent could be built and evaluated end to end. **Security testing is out of scope**; this thesis is about functional testing. The applications are small to medium web applications with seeded faults. Claims about production sites or real-world fault rates are outside what the evidence supports, and Chapter 6 states the limits on generalisation.

1.7 Organisation of the thesis

Chapter 2 reviews RL web and GUI testing, the deep RL and exploration methods used here, LLM-derived rewards, test oracles and evaluation statistics. Chapter 3 describes the system around the tester: the browser environment, the action registry, the verified-fault oracle and the product layer. Chapter 4 presents the first-generation deep RL tester and the experiments that tested it. Chapter 5 presents the hybrid tester designed from those results, and its development rounds. Chapter 6 describes the test applications and the evaluation methodology. Chapter 7 reports the evaluations. Chapter 8 concludes.

Chapter 2

Background and Related Work

This chapter positions the work. Section 2.1 covers automated web testing and its oracles, Section 2.2 RL for web and GUI testing, Section 2.3 LLM-driven testing and LLM-derived rewards, Section 2.4 the deep RL and exploration methods the testers are built from, and Section 2.5 how such tools are evaluated. Section 2.6 states the gap this thesis addresses. Statements about what a paper did or found are taken from its abstract or full text; paragraphs headed *For this work* are our reasoning.

2.1 Automated web testing and test oracles

An automated web tester drives a browser through an application and decides, at each page, what to do next. Crawljax established the model-based approach for JavaScript-heavy applications: it infers a state-flow graph of user-interface states by firing events and comparing the resulting DOM. Model-based and search-based testers explore systematically, while random ("monkey") testers are cheap and, in GUI testing, a notoriously strong baseline.

Exploring is only half of testing. The other half is deciding whether what happened was wrong: the oracle problem. Barr et al. [3] classify oracles as specified (a specification says what is correct), derived (from artefacts such as earlier versions), implicit (generic failures such as crashes and exceptions) and absent (left to a human). RL web testers use implicit oracles, namely errors in the browser console [1, 2]. These are cheap and general, but blind to wrong content that renders without error.

A *seeded-fault oracle* is a specified oracle for a known set of faults. It provides ground truth, which evaluation methodology in fuzzing prefers: Klees et al. [6] recommend measuring against known bugs, not heuristic proxies, and Magma inserts real bugs into real software with instrumentation so that bug-centric metrics do not depend on the tool. The cost is validity. Just et al. studied 357 real faults in five Java programs and found mutant detection significantly correlated with real-fault detection, but 17% of real faults were not coupled to any mutant. Seeded faults are a supported but incomplete proxy.

Once a failure is found, delta debugging [4] minimises the failing input while preserving the failure. Its original case study reduced 95 user actions that crashed a browser to 3; it is used in

the same way here, to shorten a reproduction trace.

2.2 Reinforcement learning for web and GUI testing

Web. WebExplor [2] is the reference point. It uses curiosity-driven RL to generate action sequences, with tabular Q-learning and a curiosity reward computed from a visit-count table over transitions, and builds an automaton online that sends the agent back to under-explored states. Training and testing are intertwined: the policy is learned on the application under test. Its failures are console errors, and the authors note that this oracle may miss UI bugs. WebQT changes the reward to encourage interaction like a human tester's and treats pages with the same function as one state, reporting higher code coverage on seven open-source applications. QExplore combines tabular Q-learning with a guided search and text input generation.

WebRLED [1] combines a deep Q-network with a grid-based action-value scheme, a learned action discriminator, and a curiosity reward that considers novelty both within an episode and over the global history. Liu et al. train a DQN or PPO agent to guide a crawler towards higher branch coverage. MARG runs several Q-learning agents that share Q-table data; its successor WebCQ moves to deep multi-agent RL and builds a vector for each candidate action, concatenated with the state vector, because "the available UI events vary across states". WebCQ is the closest architectural neighbour of this thesis's first tester.

A 2026 empirical study [7] compares model-based, RL-based and LLM-based web testers under six state abstractions. No single strategy excels on every dimension; RL-based approaches most often achieve the highest coverage; compact state abstractions suit RL; and code coverage is only weakly correlated with the ability to reveal failures.

Mobile. Q-testing uses a curiosity reward from a neural state-comparison module. ARES reports that deep RL achieves higher coverage and fault revelation than baselines including Q-testing. DinoDroid [8] is the clearest precedent for learning across applications: a deep Q-network learns a behaviour model from a set of existing apps and is used on new ones, evaluated by two-fold cross-validation over 64 apps (32 for training in each fold). Humanoid [9] learns from human interaction traces and reports higher coverage than six other generators, but its authors add that the learned model "was not the main reason" for the improvement.

For this work. The RL agent design is not new in itself. Count-based curiosity is the default reward in this line of work, per-action scoring over a variable action set is published, and pretraining on some applications to test others has been done on Android [8]. What differs here is the oracle (verified seeded faults instead of console errors), the outcome measure (time to the first verified fault instead of coverage), and the question asked of the learner (does it transfer, and does it use context).

2.3 Language models in testing and as reward

LLMs as testers. GPTDroid frames mobile GUI testing as a question-answering loop with an LLM. DroidAgent sets its own task goals and pursues them, reporting 61% activity coverage against 51% for earlier techniques on 15 apps. AXNav executes natural-language accessibility tests, and VETL uses a vision-language model for text inputs and element selection on the web, with exploration guided by a multi-armed bandit. The 2026 study tempers this line: its LLM-based web tester completed three to four times fewer actions in the same time budget, because every decision invokes the model, and it repeated failed or invalid actions [7]. Web-navigation benchmarks show how hard it is to act reliably on unseen sites: in WebArena the best GPT-4 agent reached 14.41% task success against 78.24% for humans, and on Mind2Web step success fell from 52.0% on unseen tasks to 38.9% on unseen websites.

LLMs as reward. Kwon et al. prompt an LLM as a proxy reward function. ELLM rewards an agent for achieving goals an LLM suggests, motivated by the observation that novelty bonuses help little where most novelty is irrelevant. Motif distils LLM preferences into an intrinsic reward; Eureka and Text2Reward have the LLM write reward code. Vision-language models have served as success detectors and zero-shot reward models, whose failures trace to the model's capability limits. Klissarov et al. find that LLMs are better used to build reward models than to act directly.

Shaping. Ng, Harada and Russell prove that adding a reward equal to the discounted difference of a potential function over successive states, $\gamma\Phi(s') - \Phi(s)$, leaves the optimal policy unchanged. Wiewiora shows the equivalence to initialising Q-values from the potential, and Devlin and Kudenko extend the guarantee to potentials that change during learning. LLM-derived potentials already exist: Lin et al. use score differences from an LLM-trained state scorer as a potential, and report that with weaker models a direct reward outperformed the potential form. Müller and Kudenko learn the potential from preferences given by small vision-language models.

For this work. Policy invariance concerns the optimal policy. It says nothing about behaviour within a finite budget, which is what a tester is measured on. The evidence for LLM rewards comes from games, robotics and synthetic web tasks; none of the retrieved works evaluates an LLM-derived reward for testing. A small, local judge is on the weak side of the capability evidence.

2.4 Deep reinforcement learning and exploration

Value-based deep RL. Q-learning learns the value $Q(s, a)$ of taking action a in state s and acting well afterwards, by bootstrapping from the next state's value. DQN [10] approximates Q with a neural network. Double DQN [11] reduces DQN's overestimation by decoupling

action selection from evaluation. Dueling networks separate state value from action advantage and give better estimates when many actions have similar values, which fits pages with many near-equivalent links. Prioritised replay replays informative transitions more often, and Rainbow combines six such extensions, including multi-step targets. These components were validated mainly on Atari, and Henderson et al. show how variance makes deep RL improvements hard to judge.

Variable action sets. A web page offers a different set of actions on every page. Dulac-Arnold et al. embed actions in a continuous space so that a policy can generalise over them. The deep reinforcement relevance network [12] embeds state and text action separately and combines them to approximate Q; this is the direct ancestor of the action-conditioned network $Q(s, \phi(a))$ used here. Jain et al. treat generalisation to unseen actions as its own problem, needing dedicated training objectives. Sentence embeddings from compact distilled encoders supply action-label features.

Exploration. Count-based exploration rewards rarely visited states: pseudo-counts generalise counting to large state spaces, and hashing-based counts work with simple hash functions, provided the hash has the right granularity. That is the same sensitivity to state abstraction that the web-testing study measured [7]. Random network distillation derives a novelty bonus from prediction error. Go-Explore [13] attributes poor exploration to two failures: forgetting how to reach promising states, and not returning to a state before exploring from it. It remedies both by keeping an archive of visited states and returning to them first.

Bandits. A multi-armed bandit chooses repeatedly among options with unknown payoff. Thompson sampling [14] keeps a posterior over each option's payoff, samples from it, and picks the best sample, which balances trying uncertain options against exploiting good ones. With a Beta posterior over a success probability, it needs only two counts per option, so it can learn within the few dozen decisions of a single test run.

Generalisation. Cobbe et al. find that agents overfit to surprisingly large training sets, and Procgen shows that diverse training distributions are essential. The survey by Kirk et al. treats the number of training contexts as a core property of an evaluation protocol and suggests fast online adaptation as a direction. With five training applications per fold, the literature predicts weak transfer.

2.5 Evaluating testing tools

Arcuri and Briand found that software-engineering studies of randomised algorithms often did not account properly for randomness, and argue for enough runs and rigorous statistical tests. Klees et al. [6] examined 32 fuzzing papers and found problems in every evaluation, recommending multiple trials, statistical tests, ground truth and a diverse benchmark; a

follow-up over 150 papers found the guidelines still widely ignored. In deep RL, Agarwal et al. recommend stratified-bootstrap interval estimates over point estimates.

When a tool may not find a bug within the time allowed, time to bug is a right-censored outcome. Survival analysis handles this: the Kaplan–Meier estimator estimates the probability of not yet having found the bug at each time, and the restricted mean survival time (RMST) averages the time to event up to a horizon. Magma applies Kaplan–Meier to time to bug, and Herrera et al. report RMST with confidence intervals and compare with the log-rank test. None of the GUI-testing papers reviewed uses censoring-aware analysis: WebExplor averages 15 thirty-minute runs and applies a Mann–Whitney U test [2].

2.6 Research gap

Within the literature reviewed:

1. **Ground-truth evaluation of RL web testers.** Web RL testers are scored on coverage and console errors [1, 2]. No retrieved work evaluates a web-testing policy, or its transfer to held-out applications, against a deterministic oracle for seeded faults.

2. **An LLM-derived reward in testing.** LLM rewards and potentials exist for games and control, and LLMs in GUI testing act as the policy. Using an LLM only to shape a cheap learned policy is untested.

3. **Censoring-aware, fault-centric evaluation in GUI testing.** Time to the first verified fault with right-censoring imports fuzzing's method into a field that reports averaged coverage, which is weakly related to failure discovery [7].

4. **Whether learning is what helps.** Humanoid's authors found their learned model was not the main cause of their gain [9]. Separating the contribution of learning from that of structure needs ablations against strong non-learning baselines.

The contribution of this thesis is the combination and the evaluation discipline. The agent architecture, the curiosity reward, the archive and the LLM potential each have direct precedents and are not claimed as novel.

Chapter 3

System Overview and the Verified-Fault Oracle

This chapter describes the parts shared by both generations of the tester: the browser environment and its action space (Section 3.1), the oracle that decides what is a fault (Section 3.2), and the product layer that turns an application into a test run and a report (Section 3.3). Figure 3.1 shows how they fit together. The explorer in the middle is the part that differs between v1 (Chapter 4) and v2 (Chapter 5).

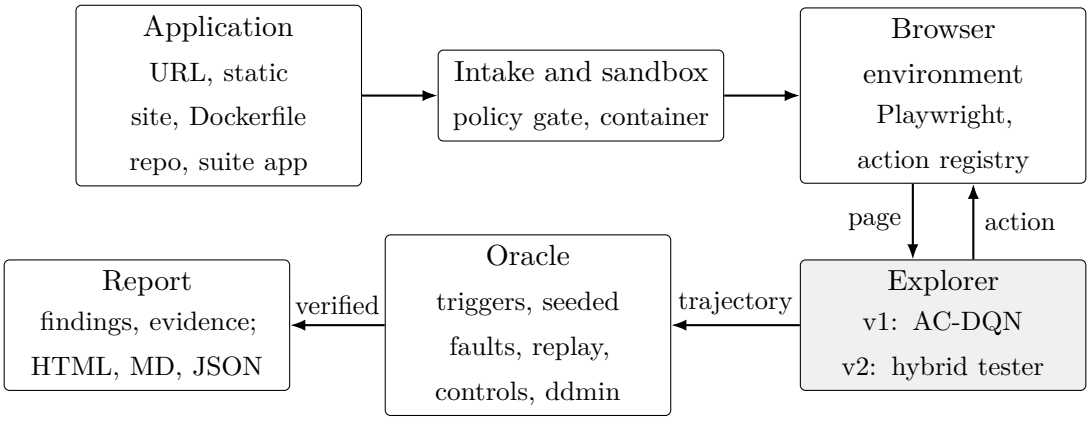


Figure 3.1: The system around the explorer. Only the explorer differs between v1 and v2.

3.1 The browser environment

The environment exposes a live application through Playwright and headless Chromium. Each *episode* starts in a fresh browser context from the application's entry page, so cookies and local storage do not leak between episodes. For applications behind a login, a scripted sign-in macro runs at the start of every episode; it is outside the agent's decisions but inside the interaction budget. Server state is reset between runs, and for the database-backed suite applications each browser context gets its own data partition (Chapter 6).

After every action the environment waits for the page to settle. It watches the DOM with a mutation observer and treats the page as settled after a quiet period, rather than waiting for

the network to go idle, which applications that poll or hold a websocket never satisfy.

3.1.1 Action registry

At each step the registry scans the page and builds the set of available actions, at most 100. It combines page-independent actions (no-op, scroll, resize the viewport, browser back, forward and refresh) with actions derived from the page's interactive elements:

- **click** on links, buttons and toggles, and **rapid click** (a double submit) on buttons;
- **type** into text fields, with one value from each of five categories: a valid typical value, the boundary minimum, the boundary maximum, an empty string, and a value of the wrong type;
- **select** an option of a dropdown;
- optionally, **custom clickables**: elements made interactive by ARIA roles, click handlers, tab index or a pointer cursor, which plain HTML scanning misses.

When a page has more candidates than slots, the budget is filled in priority order: fixed actions, then clicks (needed to progress any flow), then valid typed values (so forms can be submitted), then selects and rapid clicks, then the boundary and invalid variants. Elements visible in the viewport are allocated before those below the fold. Actions are identified by stable selectors (an element's id or name, else a structural path), so the same control keeps its identity across visits.

3.1.2 Deterministic triggers

On every step the environment records generic failure signals: console errors, uncaught page errors, unexpected HTTP failures (4xx and 5xx responses to the application's own requests), a page that never settles, and broken navigation (a navigational click that leaves the URL unchanged). A link back to the page the agent is already on is not navigational, so it does not count as broken. These triggers are the implicit oracle that published RL web testers use. Here they feed the reward and the crash-type faults; on their own they are never reported as verified findings.

3.2 The verified-fault oracle

3.2.1 Seeded faults and assertions

Every evaluation application is written with a list of *seeded faults*. Each fault is a literal source patch: an anchor string that must occur exactly once in a file, and its replacement. Applying every patch to the clean source gives the *faulty* build that agents test. For each fault F there is also a *control build*: the faulty build without F's patch.

Each fault declares how it shows itself, as an assertion over the recorded trajectory. The assertion language is deliberately closed, with eight kinds:

- `text_equals`, `text_contains`, `text_absent`: a named page region must equal, contain or not contain a literal;
- `echoes_input`: a value the agent typed earlier must appear on a later page (the core check for corrupted state);
- `numeric_equals`: a number on the page must equal an arithmetic expression over recorded inputs (for example, a total over quantities);
- `url_matches`: the step's URL must match a pattern (for broken transitions);
- `reached_with_invalid_input`: a guarded state was reached although the input that guards it was invalid;
- `region_empty`: a region the application's contract says must carry a value is empty.

Regions are named in the application's manifest, so an assertion never contains a CSS selector. Assertions are evaluated on the recorded pages without a browser, so checking is a pure, reproducible function of the record. The development suite added four extension kinds: a region stable across reload, an input absent after submission, arithmetic between regions, and `console_or_request_error` for crash-type faults (Section 3.2.3).

3.2.2 From candidate to verified finding

A violation of a fault's assertion during exploration is only a *candidate*. Verification has three steps:

1. **Replay.** The trajectory is replayed twice in fresh browser sessions against the faulty build. The violation must recur both times.

2. **Per-fault control.** The same trajectory is replayed against the control build without F. If the violation still occurs, it is not caused by F (another fault or the application's normal behaviour produces it), and the candidate is rejected.

3. **Minimisation.** Delta debugging [4] removes steps while the violation still reproduces, giving a short reproduction.

Only candidates that pass all three are *verified findings*. The rule makes false positives measurable: a run on the *clean* build has no seeded faults, so any verified finding there would be a false positive. Every evaluation in this thesis includes such clean-build control runs, and none produced a verified finding.

3.2.3 Crash-type faults

The external benchmark (Chapter 7) counts failures as console and request errors, which seeded assertions over page content do not capture. A subset of the development applications therefore also seeds *crash-type faults*: patches that make a specific interaction raise an uncaught script error or a failing request. Their assertion, `console_or_request_error`, matches the error's

signature. Verification is the same: the error must recur on replay and be absent from that fault's control build, so a generic console error elsewhere in the application is not attributed to the fault.

3.2.4 Specification-free invariants

Seeded assertions verify only what was seeded. For arbitrary applications, the v2 tester also checks three invariants that need no specification: a value the user entered must survive a reload; it must not come back altered; and the same field on the same page must not be rejected once and accepted another time. Their candidates are replayed twice and attributed to seeded faults through the control builds. They are reported separately and never counted as verified seeded faults.

3.3 The product layer

Intake and sandbox. An uploaded repository must contain a Dockerfile. Its build definition is parsed and passed through a security policy gate before anything runs. The gate refuses privileged containers, host mounts, host networking and Compose features that would merge in a second, unvalidated file. The container then runs with all capabilities dropped, a read-only root filesystem, no new privileges, CPU, memory and process limits, and an internal-only network.

Test sessions and reports. A test session runs the chosen agent within an interaction budget, verifies candidates as above, and writes a report with three parts: verified findings with minimal reproductions; unverified candidates; and, separately, any observations from the language model, marked unverified. Page bodies and frames are stored content-addressed as the run proceeds, so every finding can be traced to what produced it. Reports are available in HTML, Markdown and JSON.

Interfaces. A command-line tool tests a URL, a static site, a Dockerfile repository or a bundled application. A web interface (FastAPI with server-rendered templates) offers the same as a guided flow. The user picks a demo application, enters a URL or uploads an archive; chooses an agent (the RL agents or the random baseline) and a budget; watches a live event stream of the run; and opens or downloads the report. A research page shows every evaluation and development round, read live from the result files. The interface binds to the local machine by default, because it builds and runs uploaded code.

Chapter 4

A Deep Reinforcement Learning Tester (v1)

The first-generation tester (v1) is a deep RL agent. This chapter formulates the testing problem as an MDP (Section 4.1), describes the agent (Section 4.2) and its reward, including the LLM progress term (Section 4.3), and reports the three studies that tested it: a pre-registered transfer experiment (Section 4.4), a pre-registered contextual diagnostic (Section 4.5), and checks of the LLM components (Section 4.6). Section 4.7 draws the conclusions that shaped v2.

4.1 Testing as a Markov decision process

An *episode* is a walk of at most 40 actions from the application's entry page. A *run* (or hunt) is a sequence of episodes on one application until a budget of $B = 2{,}000$ browser interactions is spent. The state s_t is a summary of the episode so far and the current page; the actions are those the registry offers on the current page (Section 3.1), so the action set changes at every step; and the reward is described in Section 4.3. The agent never sees the seeded faults or the oracle's verdicts: learning from them would leak the answer key into the policy.

State. The state vector has 25 numbers: the episode's history (step, distinct pages, recent repetitions), the page's composition (counts of links, buttons, fields, forms, by kind), how much of the current form is filled, and the history of the LLM's progress judgments. It deliberately carries no page text, heading or URL.

Action features. Each candidate action a is described by a 95-dimensional vector $\phi(a)$: its type; a sentence embedding of its label (MiniLM, reduced by PCA); its role inferred from markup (submit, navigate, edit, remove and so on); flags such as being in the viewport or inside a form; the category of value to type; and how often it has been taken in the episode.

4.2 The agent: an action-conditioned dueling double DQN

A fixed output head, one unit per action slot, cannot work here: slot k means a different control on every page. The agent instead scores each (state, action) pair with one shared network, $Q(s, \phi(a))$, following the relevance network of He et al. [12]. The state is encoded by a 64-unit layer and each action by a 96-unit layer; a 128-unit scorer combines them into an advantage, and a separate state-value head makes the network dueling. Because one scorer is shared by every action, the network is permutation-equivariant over the action set and accepts any number of actions.

Training uses Double DQN targets [11], 3-step returns with $\gamma = 0.95$, and prioritised replay ($\alpha = 0.6$, $\beta_0 = 0.4$, a buffer of 30,000 transitions, batches of 64, learning rate 3×10^{-4}, target update every 200 steps).

Search-guided exploration. With uniform ϵ-greedy exploration, pretraining never reached the order-confirmation pages of the development applications, so the reward for completing a task never paid. Exploratory actions are therefore taken, with probability 0.75, from a fixed search priority (the same linear score over the action features that the `search_S` baseline uses), and otherwise uniformly at random. On the held-out application every learning arm uses one ϵ schedule (0.50 decaying to 0.05 over half the budget), so arms do not differ in how much search they receive.

Online adaptation. For transfer, the pretrained network is kept and a small residual head, initialised to zero, is added and trained on the new application. The pretrained agent therefore starts exactly where pretraining left it and adapts from there.

Bandit variant. The same network with $\gamma = 0$ (no bootstrapping) is a contextual bandit. It tests whether multi-step credit assignment adds anything.

4.3 Reward

The reward has an environment part and an optional LLM part:

$$r_t = r_t^{\text{env}} + \lambda\, r_t^{\text{LLM}}. \tag{4.1}$$

Environment terms. A small step cost; a novelty bonus that decays with the visit count of the abstracted state; a bonus for revealing new controls, capped at one control per step so that a single-page application does not dominate the scale; a repetition penalty; and one-time bonuses for the deterministic triggers (console error, failed request, broken navigation). The total is clipped to $[-6, 6]$.

The LLM progress term. A local LLM (`qwen2.5:7b-instruct`, temperature 0) reads a text rendering of each newly reached page and rates how far it is through a task, as a potential $\Phi(s) \in [0, 1]$. Judgments are cached by page content, so each distinct page is judged once. The first design was potential-based shaping, $\lambda(\gamma\Phi(s') - \Phi(s))$. It preserves the optimal policy, and that is why it failed: once novelty is exhausted, nothing in the optimal policy reaches outcome pages, and an agent pretrained on one application never reached its confirmation page in 1,500 steps. The term actually used is an *episodic progress record*,

$$r_t^{\mathrm{LLM}} = \max\Big(0,\ \Phi(s_{t+1}) - \max_{k \le t} \Phi(s_k)\Big), \tag{4.2}$$

which pays only for a new maximum within the episode, so revisiting or looping earns nothing and the total per episode is at most 1. A sparse *outcome bonus* of 3 pays the first time in an episode that a page the model places in the "outcome displayed" stage is reached, because the model rated any form page highly and completing a task added almost nothing. Neither term sees the seeded faults.

4.4 Experiment 1: does pretraining transfer?

Question. Does pretraining on five applications make verified-fault discovery on a held-out sixth faster than learning from scratch? The comparison is against information-matched non-learning baselines, with and without the LLM reward.

Design. The design was pre-registered: hypotheses, arms, analysis code and decision rule were committed before any result existed. It is leave-one-application-out over the six benchmark applications (Chapter 6); four applications were never used during development (the *pristine* folds). Ten arms ran (Table 4.1): uniform random; two fixed search heuristics (`search_A0`, hand-weighted priority; `search_S`, the feature-based score that guides exploration); the DQN from scratch and pretrained, each with and without the LLM reward; the pretrained DQN frozen (zero-shot); and the bandit from scratch and pretrained. Pretraining used 1,500 steps per training application. Each hunt had $B = 2{,}000$ interactions, with seeds 0–2 and a mechanical top-up rule. The outcome was T, interactions to the first verified fault, right-censored at B. The primary statistic was the per-fold paired difference in restricted mean T, summarised by its median across folds with a fold-level bootstrap interval; the stratified log-rank test was secondary.

Execution. There were 291 jobs and 0 failures. The LLM arms ran on a local RTX 4050 GPU and the other arms on four AWS spot instances (about US$1.6). The analysis was exactly the pre-registered one.

Table 4.1: Experiment 1: discovery per arm over all folds (runs of 2,000 interactions).

Arm	Runs	Found a fault	Rate	Restricted mean T
`dqn_pretrained_nollm`	18	7	0.39	1,402
`bandit_pretrained`	24	10	0.42	1,483
`dqn_pretrained`	27	9	0.33	1,520
`dqn_scratch`	27	9	0.33	1,610
`dqn_zeroshot`	18	4	0.22	1,664
`bandit_scratch`	24	5	0.21	1,727
`dqn_scratch_nollm`	18	5	0.28	1,748
`search_S`	18	3	0.17	1,844
`random`	18	2	0.11	1,947
`search_A0`	18	1	0.06	1,953

Table 4.2: Experiment 1: pre-registered comparisons (negative Δ = faster).

Comparison	Median Δ	95% interval	Log-rank p
`dqn_pretrained` − `dqn_scratch` (primary)	0	$[-749, +290]$	0.527
`bandit_pretrained` − `bandit_scratch`	−280	$[-732, +188]$	0.086
`dqn_pretrained` − `search_S`	−169	$[-884, 0]$	0.087
`dqn_pretrained` − `search_A0`	−170	$[-1{,}212, 0]$	0.006
`dqn_pretrained` − `random`	−151	$[-1{,}212, 0]$	0.022
LLM reward on − off (pretrained DQN)	+17	$[-122, +378]$	0.754
Adaptation on − off	−84	$[-430, 0]$	0.534

Results. Pretraining gave no measurable benefit. The primary comparison has a median difference of 0 interactions, with an interval reaching far on both sides; on the four pristine folds it was −310 ($p = 0.66$). The LLM reward made no detectable difference. Learned arms found a fault in more runs than search or random (0.33–0.42 against 0.06–0.17), but their fold-level intervals reach zero and the effect rests largely on one development fold. Faults 9–16 contextual steps deep, on two applications, were never found by any arm. The oracle produced no verified finding on any clean build.

A post-hoc description of the hunts showed that about 84% of each learned arm's actions were the network's own greedy choice. The null result is not the exploration guide masking the network: the learned Q drives behaviour, yet pretrained and scratch DQN find faults in the same share of runs (9 of 27 each). What helped was learned inside the application under test. This matches the literature's prediction for five training applications per fold [8].

4.5 Experiment 2: context or memorisation?

Question. Does the network rank an action by the state it is in, or has it memorised actions, words or layouts?

Design. This experiment was also pre-registered, with separate applications, runs and report. A seeded generator produced 12 variants of one request workflow. They vary the vocabulary (training words; synonyms, unseen words and pseudo-words held out with zero token overlap), the structure (training layouts; tabs, a modal and reordered steps held out) and add a decoy control. Four agent arms differ only in which inputs the network sees: the full agent; a label-blind agent; a lexical-only agent; and an action-only agent that never sees the state. Probe sets of states, with each action's effect labelled by actually executing it, were fixed before any arm ran. There were 228 jobs (12 pretrainings, 216 hunts) and 0 failures.

Table 4.3: Experiment 2: pre-registered verdicts for the full agent.

Hypothesis	Deciding numbers	Verdict
H-ctx: state–action contextual learning	within-label AUROC 0.51 $[0.44, 0.59]$; rule ≥ 0.70	**not supported**
H-act: action memorisation	decoy probe rate 0.31 $[0.00, 0.91]$	inconclusive
H-lex: lexical memorisation	top-1 ratio on unseen vocabulary 0.88 $[0.35, 1.18]$	inconclusive
H-app: application memorisation	top-1 ratio on new structures 0.63 $[0.44, 0.88]$	inconclusive

Results. Contextual learning was not supported. Within a label, the network's ranking of an action barely depends on whether that action helps in the current state (AUROC 0.51, chance level), and only 7% of the variance of Q came from the label $\times$ state interaction: the state moves every action's value together and almost never reorders them. The agent does rank useful actions above chance overall (top-1 accuracy 0.56, against 0.18 for random); this held up under vocabulary changes but not under every layout change (tabs 0.35, modal 0.58). On the eight held-out variants, the arms that see the state found the fault in 19–20 of 24 hunts, against 2 of 24 for search and random, and the label-blind arm did as well as the full one. The memorisation hypotheses stayed inconclusive with three seeds: on the decoy, one seed nearly always took it and the other two never did.

4.6 Checks of the LLM components

Is the progress reward rewarding the right thing? This post-hoc, descriptive check asked whether the LLM's progress score separates pages where a fault becomes observable from pages a random walk visits. On the two development applications, where the prompt was

tuned, it separated them perfectly (AUROC 1.00). On the four pristine applications it did not (0.73, 0.52, 0.47, 0.38). The score rewards the "happy path", while many faults need deviations (invalid input, an edit after review), and most of what it paid duplicated the novelty bonus. This agrees with the warning that small LLM judges make noisy reward models.

Can the LLM label pages reliably? For v2, an LLM labeller was to classify page kinds and control roles once per page. It had to pass gates fixed in advance: page-kind accuracy ≥ 0.85 and control-role accuracy ≥ 0.80.

Table 4.4: LLM labeller accuracy against ground truth (gates 0.85 / 0.80).

Test	Page kind	Control role	Gate
Stage 0, generated variants	0.68	0.78	failed
Revised labeller, 69 fresh pages	0.61	0.75	failed
Revised labeller, on its own revision data	0.93	0.83	(would have passed)

Both gates failed. The last row shows why the re-test had to use fresh pages: judged on the data it was revised from, the labeller would have been adopted.

4.7 What v1 taught

Table 4.5: Findings from v1 and the design consequences for v2.

Finding	Consequence for v2
Pretraining did not transfer; learning inside the application helped	keep learning, but online, inside each application
No state–action contextual learning	make the state explicit: page kind, task stage, form status
The LLM reward is misaligned with faults; the labeller failed its gates	remove the LLM from the reward and from the decision path; label pages from markup
Deep faults were never reached	explore by remembering states and returning to them (Go-Explore)
Faults need deviations from the happy path	deviate deliberately, with test-design operators

Chapter 5

The Hybrid Tester (v2)

The second-generation tester (v2) follows from the conclusions of Chapter 4. It makes the page state explicit instead of hoping a network learns it; it remembers reached states and returns to them exactly, so depth does not depend on luck; it deviates from the happy path deliberately, with operators from standard test design; and it keeps learning light and online, inside the application under test. Figure 5.1 shows the loop. Sections 5.1–5.4 describe its parts, Section 5.5 the options added for the final tuning round, and Section 5.6 the development rounds that shaped it.

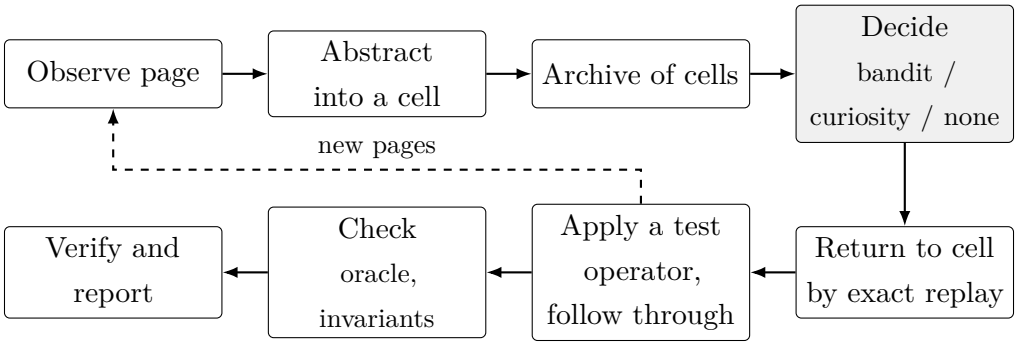


Figure 5.1: The hybrid tester's loop. Every replayed action counts towards the budget.

5.1 Page abstraction

Each page becomes a small symbolic *cell*, computed deterministically from the markup:

- the page kind, from a closed vocabulary (home, list, detail, form, review, outcome, info, login, error, other);
- the URL path with identifiers replaced (`/orders/{n}`);
- the main heading;
- whether the form on the page is empty, partly filled or full;
- whether an error or alert is shown;
- whether the page offers a submit action.

Controls get roles from markup cues: submit, edit, remove, add, back, navigate. Two rules came from development evidence. Links and buttons in the page's navigation, header or footer are never "submit": they start a new journey rather than complete the current one. And role cues match word prefixes, so "Saving" and "Save" share a role.

Experiment 2 showed that the network does not learn these distinctions by itself, and the literature finds that the choice of state abstraction drives results in both web testing [7] and count-based exploration. The granularity was tuned on development applications only: exact field combinations gave 150–225 cells on one application, so form state was reduced to empty, partial and full, which gave about 20.

5.2 Archive with exact return

The tester keeps an archive of every cell it has reached, with the shortest known action sequence to it, its visit count, and the operators already tried there. At the start of an excursion it chooses a cell with probability proportional to a weight

$$w(c) = \frac{1}{\sqrt{1+n_c}} \cdot \left(1 + \frac{\min(u_c, 10)}{5}\right) \cdot \frac{1}{1+2f_c} \cdot \rho_c \cdot \left(1 + \beta_d \frac{\min(d_c, 30)}{30}\right), \tag{5.1}$$

where n_c is the number of visits, u_c the number of untried operators, f_c the number of failed returns, ρ_c is 1 if enough of the episode remains after reaching the cell and 0.1 otherwise, d_c the cell's depth, and $\beta_d = 2$ a preference for depth (faults sit many steps into a workflow). The tester then *returns* to the cell by replaying its path exactly, and acts from there. This is Go-Explore [13]: poor exploration comes from forgetting how to get back to promising states, and the deep, unfound faults of Experiment 1 were exactly that case.

Paths are pruned of steps that change neither the page nor the cell, and every replayed action counts towards the interaction budget, so returning is not a hidden advantage. One replay bug, in which pruning removed steps that raised a notice the next step depended on, was found on a development application and fixed; failed returns fell from 6 of 14 to 0 of 28.

5.3 Test operators

From a cell the tester applies a *test operator*, a short procedure from standard test design:

- `complete_step`: fill the form validly and submit (the path deeper);
- `choose_option`: take each option of a dropdown in turn (equivalence classes);
- `invalid_input`: enter an empty value, a value of the wrong type, or a boundary minimum or maximum (boundary-value analysis);
- `edit_after_review`: reach a review step, go back and change an earlier answer (state-transition testing);
- `back_and_resume`, `reload_and_resume`: interrupt a flow and continue;
- `double_submit`: submit twice in quick succession;

- `collection_churn`: add and remove items in a collection;
- `combine` (development option, not adopted): two deviations on one form, such as a dropdown option together with an invalid field (pairwise testing);
- `browse_list` (option, Section 5.5): sort or filter a list, page forward, and open an item on a later page.

Each operator then *follows through*: it continues the task to submission, for up to four submissions, because a deviation usually shows its effect only on a later page. When a page offers several submit actions (for example, "Update bag" and "Go to checkout"), operators take them in turn on repeat visits. Operators are chosen by technique, not from any application's faults. They are how the tester leaves the happy path, which the LLM progress reward could not do.

5.4 The decision layer

The decision layer chooses which cell to return to and which operator to apply on which target. Three variants exist; each is an arm.

No learner (`hybrid_nolearn`). Cells by the archive weight; at a cell, untried (operator, target) pairs first, in random order. This is the non-learning reference.

Thompson-sampling bandit (`hybrid_bandit`). A Bayesian bandit learns, online inside the application, which operator pays off on which kind of target. Its keys are semantic and back off from specific to general:

$$(\text{page kind}, \text{operator}, \text{target type}) \;\rightarrow\; (\text{operator}, \text{target type}) \;\rightarrow\; (\text{operator}).$$

Each key has a Beta(α, β) posterior over the probability that the operator produces an interesting outcome. A specific key's prior is the more general keys' evidence, discounted by 0.3, so a new combination starts from what similar ones did. At each decision the tester samples from every candidate's posterior and takes the best sample [14]. The reward is the tester's own observation, never the seeded-fault oracle: an alert shown, new cells reached, or an invariant candidate. A bandit was chosen over a deep network for three reasons: Experiment 1 found no gain from bootstrapped multi-step learning; a 2,000-interaction run makes only about 50–135 decisions, enough for a bandit over a few dozen interpretable keys but far too few for a network; and keys based on page kinds, not application text, can carry across applications.

Curiosity Q-learning (`hybrid_curiosity`). This variant adopts the literature's design for the free exploration steps between operators. Following WebExplor [2] and WebRLED [1], a

tabular Q-learner over cells is rewarded on every step for curiosity:

$$r = \frac{1}{\sqrt{N(c,a,c')}} + 0.5 \cdot \mathbb{1}[c' \text{ not yet seen this episode}], \tag{5.2}$$

where $N(c, a, c')$ counts the transition. It learns with $\alpha = 0.5$, $\gamma = 0.9$ and $\epsilon = 0.1$, with untried actions initialised optimistically at $R_{\max}/(1-\gamma)$, so that an action that changes nothing cannot outrank untried ones by bootstrapping off itself. Being off-policy, it learns from every transition, about 2,000 updates per run, against about 135 outcomes for the bandit. A standalone tester built only from these components (no operators, no bandit) was also run, as `dev_curiosity_rl`.

5.5 Options for the final tuning round

After the held-out evaluation (Chapter 7) and the rebuild of the development applications, three options were added, each switched on in its own tuning arm:

- `browse_list` (`dev_v3_browse`). The held-out evaluation exposed a blind spot: the operators work on forms, and no operator pages through a list. This operator applies a sort or filter control, presses "next" or "load more" up to twice, and opens an item on the later page.
- **Custom clickables** (`dev_v3_clickables`). The registry also offers elements made clickable by ARIA roles, click handlers or styling, such as kanban cards and tab strips.
- **Failure feedback** (`dev_v3_failfeedback`). Cells where a browser error followed an action get extra archive weight (up to $\times 3$), since code that already misbehaves is likelier to misbehave again nearby.

The curiosity variant (`dev_v3_curiosity`) is the fourth tuning arm.

5.6 Development rounds

The design was tuned on development data only: the six benchmark applications, a generated application family, Gitea, and the eight development suite applications (Chapter 6). Each round compared arms on the same applications and seeds, and ended with a recorded decision. Table 5.1 lists them.

Table 5.1: Development rounds of the hybrid tester and their decisions.

Round	Tested	Decision
Benchmark 1	bandit, no learner, random (4 apps)	discarded (code changed mid-round); replay bug found and fixed
Benchmark 2	bandit vs no learner, with clean builds (6 apps)	validation invariant narrowed; form fixes
Benchmark 3	free-exploration variants	depth preference and weighted exploration adopted
Benchmark 4–5	which submit actions a cell records	cells record *whether* a submit is offered
Benchmark 6–7	whole-word vs prefix matching of role cues	prefix matching adopted
Suite 1	bandit vs no learner, 8 suite apps	site navigation never submits; submit actions taken in turn
Suite 2	the same after the fix, 14 apps	kept; learner not ahead
Suite 3	two corrected bandit rewards (novelty, surprise)	no difference: reward not the bottleneck
Suite 4	pairwise operator + technique-first choice	worse; not adopted
Suite 5	curiosity Q-learning, in the hybrid and standalone	neither adopted

Table 5.2: All arms on the same runs: suite rounds 2–5, 14 applications × 3 seeds. Difference is the per-application mean T against `hybrid_nolearn`, with a bootstrap interval over applications (negative = faster).

Arm	Found a fault	Median T	Mean T	Difference [95% interval]
`hybrid_nolearn`	32 / 42	742	889	reference
`hybrid_bandit`	30 / 42	335	840	-49 $[-279, +190]$
`dev_bandit_surprise`	31 / 42	586	844	-46 $[-257, +143]$
`dev_bandit_novelty`	31 / 42	486	854	-35 $[-252, +190]$
`dev_pairwise_bandit`	27 / 42	757	943	$+53$ $[-121, +227]$
`dev_hybrid_curiosity`	29 / 42	612	978	$+89$ $[-38, +202]$
`dev_pairwise_nolearn`	26 / 40	1,069	1,046	$+119$ $[-37, +281]$
`dev_curiosity_rl`	17 / 42	2,000	1,313	$+423$ $[+142, +744]$
`random`	18 / 42	2,000	1,355	$+465$ $[+178, +790]$

Is the learner helping? The top four arms in Table 5.2 are statistically tied. `hybrid_nolearn` verified a fault in the most runs, and `hybrid_bandit` was fastest when it found one. No learner variant improved on no learner: three bandit reward signals, the pairwise operator, and curiosity Q-learning. The standalone curiosity tester was fast on shallow faults but never found the deep ones, and was significantly worse than the hybrid, level with random. It is a tester built from

the published components, not a reproduction of WebExplor or WebRLED; their published results (coverage and crash-type failures over long runs) are not contradicted by it.

The design leaves the learner little to decide, for four reasons. On about 5 of the 14 applications a fault is found within the first few decisions, before learning can matter. The faults no arm finds need two deviations in one walk (for example, express delivery *and* an empty address), which choosing one operator more cleverly cannot produce. The option space is small: about 28 cells times 8 operators can mostly be covered in about 135 decisions, so trying untried options systematically already does most of the work. And operators with many targets crowd out the rest: invalid input took 30% of all decisions because pages have many fields. This agrees with the Humanoid authors' observation that their learned model was not the main reason for their gains [9]: on functional faults, the structure (abstraction, archive with exact return, operators) does the work.

Chapter 6

Test Applications and Evaluation Methodology

A tester can only be evaluated on applications whose faults are known. This chapter describes the four sets of test applications (Sections 6.1–6.5), how an application is admitted before any agent touches it (Section 6.6), how applications were split between development and evaluation (Section 6.7), and the metrics and statistics (Section 6.8). Section 6.9 states the threats to validity.

6.1 The six-application benchmark

Six small web applications were built to one design contract for the v1 experiments: an account hub, an admin records system, a booking desk, a media library, a shop catalogue and a support portal. Each carries two seeded faults, 12 in total, in five classes: calculation error, constraint bypass, persistence failure, state corruption and broken transition. Some faults sit 9–16 contextual steps deep. Two applications (the shop catalogue and the booking desk) were used during development; the other four stayed pristine until the transfer experiment.

6.2 The generated application family

For Experiment 2, a seeded generator produced twelve variants of one loan-request workflow. They differ in vocabulary, structure and decoys, and each was admitted by the same validator as the benchmark (Section 4.5).

6.3 The development suite

Eight applications were built to be genuinely different from each other and from the benchmark: different domains, architectures and stacks (Table 6.1). They were first built as zero-dependency applications. Then, at the project owner's request, they were rebuilt as full-stack applications. Each now has its own database (PostgreSQL or MongoDB), a sign-in page and a richer data

model. The five rebuilt last were also enlarged to at least 12 views, 3 entities with full create, read, update and delete, 5 interaction patterns, and 2 new faults that are not about forms.

Table 6.1: The development suite (version 2).

App	Domain and architecture	Server	Database	Faults	Crash
tidewater	personal banking; single-page app over a JSON API	Python	PostgreSQL	3	–
clinicflow	clinic front desk; server-rendered branching wizard	Python	PostgreSQL	5	–
stackroom	stockroom inventory; data grid with server-side paging, sorting, bulk actions	Python	PostgreSQL	5	2
ledgerly	expense claims with multi-role approval; server-rendered forms	Node.js	PostgreSQL	5	–
bagstreet	small shop; server-rendered pages and HTML forms	Python	MongoDB	5	–
helpline	broadband care desk; pages with HTML fragment updates	Python	MongoDB	7	2
orbit	kanban boards; modal-heavy single-page app	Node.js	MongoDB	5	2
wayfarer	trip itinerary builder; client drafts and sealed server versions	Node.js	MongoDB	6	2
				41	8

Isolation between sessions. Every signed-in browser context gets its own data partition: a schema per session in PostgreSQL, a database per session in MongoDB. This resets the application between episodes and keeps parallel runs independent. The database drivers are vendored and pinned, so a build needs no network.

Crash-type faults. Four applications also carry two crash-type faults each (Section 3.2.3), so that the tuned tester is exercised on the kind of failure the external benchmark counts. By the project owner's decision only a subset of applications has them. Orbit's were admitted (40 of 40 checks, including Docker); helpline, stackroom and wayfarer were admitted with all checks passing (44, 40 and 41 checks, including Docker).

6.4 The validation suite

Eight further applications were planned by an independent agent working only from the suite's design contract, without access to the tester's code or results, and built for the held-out evaluation (Table 6.2). Each has three seeded faults, 24 in total, and each was admitted with 20 of 20 checks. They were evaluated in their first version (without a database server or sign-in) and have not been changed since.

Table 6.2: The validation suite, used once, for the held-out evaluation.

App	Domain	Stack	Faults
dutyday	a food bank's volunteer rota	Python, SQLite	3
fosterly	an application to become a foster carer, saved section by section	Node.js, JSON files	3
larder	a personal recipe box with scaling and unit conversion	Node.js, JSON files	3
markbook	a teacher's markbook with a student view	Python, SQLite	3
quizcraft	a survey editor with undo, redo and a respondent preview	Python, SQLite	3
sideline	a football league's results, confirmations and table	Python, SQLite	3
skylark	a nature group's bird-sightings register with a paged feed	Node.js, JSON files	3
tally	a club election by ranked ballot	Node.js, JSON files	3

6.5 The external benchmark

To compare with published RL web testers on their own ground, the final evaluation uses the six subject applications of WebExplor [2], which WebRLED's first benchmark [1] also uses:

- Dimeshift, an expense tracker;
- Pagekit, a content management system, tested in its admin area;
- Splittypie, which splits bills;
- Phoenix, a Trello clone;
- Retroboard, a retrospective board;
- Petclinic, the Spring demonstration application.

They are open-source applications not written by this project. They run from the Docker images of the DIG replication material [15], which instrument the client JavaScript with Istanbul, so branch coverage is measured the way the papers measured it. To keep the comparison honest, nobody looked at these applications before the re-tuned tester was frozen and the comparison pre-registered (Section 7.3).

6.6 Admission

Before any agent runs on an application, a harness must admit it mechanically. For each fault, a scripted *witness* trace must trigger the fault's assertion on the faulty build and not on that fault's control build. The witness must replay deterministically and minimise to a short reproduction. Benign traces must trigger nothing. A random walk on the clean build must produce no candidate. The faulty build must differ from the clean build *only* by the declared patches; a diff audit checks this. The development suite adds a data-model contract (seeded record counts per entity), a breadth contract (views, entities, interaction patterns), sign-in checks, crash checks and a Docker build. An application that fails any check is not used. A

diversity check also confirms that the suite's applications do not share code (the largest overlap was 3.2%).

6.7 Data split and independence

- **Development** (used for tuning and design decisions): the six benchmark applications, the generated family, Gitea (a real open-source application, as a non-admin user), and the eight development suite applications.
- **Held-out**: the eight validation applications, used once, by the pre-registered evaluation of Chapter 7.
- **External**: the WebExplor and WebRLED subject applications, used once, by the final comparison.

The split was fixed before any agent ran on the suites. Development results are tuning data. They are never presented as evidence of generalisation, and the held-out result belongs to the version of the tester frozen before it ran.

6.8 Metrics and statistics

Outcome. For a run with budget B, T is the number of interactions until the first verified fault. Every browser action counts, including replays and returns. A run that finds nothing is right-censored at $T = B$.

Per arm. The *discovery rate* is the share of runs that verified at least one fault. The *restricted mean* of T (RMST) averages T over runs with censored runs at B. *Distinct faults* counts how many of the seeded faults an arm verified in at least one run. Clean-build control runs give the false-positive count. For the tuning round two descriptive metrics were added. *Unique failures* counts distinct browser-error signatures (console, page and failed-request errors). *JavaScript coverage* is the share of the application's client-side code executed, measured with V8's precise coverage through the Chrome DevTools Protocol, attached before the first navigation.

Comparisons. Applications differ far more than seeds do, so the *application* is the unit of inference. For two arms, the per-application difference in restricted mean T is computed on matched (application, seed) pairs; the median across applications summarises it, with a 95% bootstrap interval resampling applications. The stratified log-rank test is secondary. The pre-registered decision rule:

- *supported* if the median difference is negative (faster) and its interval excludes zero;
- *refuted* if it is positive and the interval excludes zero;
- otherwise *inconclusive*.

Pre-registration. For every pre-registered experiment, the hypotheses, arms, applications, budget, seeds, analysis code and decision rule were committed to the repository, and the commit was tagged before any agent ran on the evaluation applications [6]. Deviations, such as rerunning a job lost to an infrastructure failure, follow rules written in advance and are reported.

6.9 Threats to validity

- **Same developer.** The test applications and their seeded faults were built by the same project as the tester. Independent planning of the validation suite and the external benchmark reduce, but do not remove, this threat.
- **Seeded faults.** Seeded faults are an incomplete proxy for real faults, and the oracle sees only what was seeded.
- **Scale.** Eight applications per evaluation leave wide intervals at the application level. Small effects cannot be told apart from noise.
- **Fault difficulty.** If the held-out faults are shallow, every arm finds them and differences compress. Chapter 7 reports that this happened.
- **Tuning leakage.** Decisions made on development data can overfit it; the held-out and external evaluations exist to measure that.

Chapter 7

Evaluation

This chapter reports three evaluations of the hybrid tester. The pre-registered held-out evaluation on the validation suite (Section 7.1) is complete. The final tuning round on the rebuilt development suite (Section 7.2) and the external comparison with WebExplor and WebRLED (Section 7.3) follow it.

7.1 Held-out evaluation (pre-registered)

7.1.1 Design

The pre-registration was frozen on the project owner's approval, and the commit was tagged, before any agent ran on any validation application.

- **Applications**: the eight validation applications (Table 6.2), 24 seeded faults.
- **Arms**:
  - `random`, uniform over the page's valid actions;
  - `hybrid_nolearn`, the primary arm;
  - `hybrid_bandit`;
  - `v1_dqn_pretrained`, v1's best AC-DQN arm from Experiment 1: pretrained without the LLM reward and adapting online, using that experiment's own checkpoints. Seed s loads the checkpoint of fold s mod 6, pretraining seed s mod 3.
- **Runs**: seeds 0–4, $B = 2{,}000$ interactions, 160 hunts, plus 32 clean-build control runs (one per arm and application). The run took about seven and a half hours with four workers and no other workload.
- **Hypotheses** (difference in restricted mean T, decision rule of Section 6.8):
  - H1 (primary): the hybrid tester is faster than random;
  - H2: the bandit makes the hybrid faster;
  - H3: the hybrid with the bandit is faster than random;

– H4: the hybrid is faster than v1's best AC-DQN.

One of 192 jobs failed for an infrastructure reason: Windows held a file lock while the temporary build folder of a clean-build control was being deleted. As pre-registered, the job was rerun unchanged; it completed with no finding.

7.1.2 Results

Table 7.1: Held-out evaluation: pre-registered comparisons (negative Δ = first arm faster).

	Comparison	Median Δ	95% interval	Apps favouring	Log-rank p	Verdict
H1	`hybrid_nolearn` − `random`	−324	$[-665, +148]$	6/8	0.159	inconclusive
H2	`hybrid_bandit` − `hybrid_nolearn`	+86	$[-304, +285]$	4/8	0.781	inconclusive
H3	`hybrid_bandit` − `random`	−191	$[-801, +67]$	5/8	0.180	inconclusive
H4	`hybrid_nolearn` − `v1_dqn_pretrained`	+42	$[-196, +574]$	4/8	0.276	inconclusive

Table 7.2: Held-out evaluation per arm (40 runs each).

Arm	Found a fault	RMST	Faults per run	Distinct faults	Clean false positives
`v1_dqn_pretrained`	34 (0.85)	**595**	1.18	17 / 24	0 / 8
`hybrid_nolearn`	30 (0.75)	737	1.15	18 / 24	0 / 8
`hybrid_bandit`	34 (0.85)	769	**1.23**	**19** / 24	0 / 8
`random`	32 (0.80)	978	1.00	13 / 24	0 / 8

Table 7.3: Held-out evaluation per application: runs that found a fault (of 5) and mean T.

App	`random`	`v1_dqn_pretrained`	`hybrid_nolearn`	`hybrid_bandit`
dutyday	4/5 · 1,217	**5/5 · 281**	4/5 · 788	4/5 · 778
fosterly	5/5 · 1,262	4/5 · 763	4/5 · 681	**5/5 · 370**
larder	5/5 · 411	5/5 · 407	**5/5 · 211**	5/5 · 393
markbook	1/5 · 1,805	**4/5 · 567**	3/5 · 1,141	4/5 · 1,442
quizcraft	5/5 · 299	5/5 · 97	**5/5 · 81**	5/5 · 366
sideline	2/5 · 1,623	1/5 · 1,759	**4/5 · 620**	3/5 · 822
skylark	5/5 · 980	**5/5 · 614**	0/5 · 2,000	3/5 · 1,696
tally	5/5 · **229**	5/5 · 276	5/5 · 376	5/5 · 284

All four pre-registered comparisons are inconclusive (Table 7.1). As the pre-registration anticipated, eight applications give the median rule wide intervals, and no comparison was supported or refuted. The per-fault counts are in Appendix A.2.

7.1.3 What the held-out data shows (descriptive)

1. **The held-out faults were easier for random exploration than the development ones.** Random verified a fault in 80% of runs, against 43% (18 of 42) on the development suite. Several faults sit one or two interactions from a page that random visits constantly. Three faults were verified in 5 of 5 runs by every arm, which compresses every difference.

2. **The hybrid arms reached the deeper faults.** 22 of the 24 faults were verified by at least one arm; two by none. Of the 16 faults random verified in at most one of five runs, the two hybrid arms together verified 14 at least once, and `v1_dqn_pretrained` 9. Nine faults were verified by a hybrid arm and by no random run. This is where test operators help: completing forms validly, invalid input, edits after review.

3. **One clear blind spot: list paging.** All three skylark faults sit behind its feed's "Load more" button. Random presses it often by chance. The hybrid's operators work on forms and none pages through a list, so `hybrid_nolearn` reached the right pages in every run but never repeated the paging (0 of 5). This cost H1 one of its eight applications. It was reported, not fixed, because the code was frozen; it motivated the `browse_list` operator (Section 5.5).

4. **v1's pretrained AC-DQN did better here than on its own benchmark**: discovery 0.85 and the lowest mean time, notably on dutyday and markbook. Whether that reflects the method or the applications' shallower faults cannot be separated with eight applications (H4 inconclusive).

5. **The learner (H2).** `hybrid_bandit` found a fault in more runs than `hybrid_nolearn` (34 against 30) and more distinct faults (19 against 18), with a slightly higher mean time (769 against 737). As in development, the learner is not distinguishable from no learner.

6. **Zero false positives.** No clean-build control verified a seeded fault (0 of 32 runs).

Specification-free invariants. As pre-registered, they are reported separately. They flagged no seeded fault for any arm. On clean builds they produced three reproduced candidates (two for `hybrid_nolearn`, one for `hybrid_bandit`), all on quizcraft's editor, all of the form "an entered value is gone after reload". On inspection, quizcraft keeps its working copy in page memory until the user saves, so losing unsaved edits on reload is intended. They are false alarms: the reload invariant cannot tell saved state from unsaved state.

7.2 Tuning round v3 on the rebuilt development suite

7.2.1 Design

After the held-out evaluation, the development suite was rebuilt (Section 6.3) and the tester re-tuned on it before the external comparison. Eight arms ran on the eight development

applications with seeds 0–2, 192 runs of 2,000 interactions:

- the references: `random`, `hybrid_nolearn`, `hybrid_bandit` and `v1_dqn_pretrained`;
- one option each (Section 5.5):
  - `dev_v3_browse`,
  - `dev_v3_clickables`,
  - `dev_v3_curiosity`,
  - `dev_v3_failfeedback`.

Each run records verified faults (including crash-type faults), unique failures and JavaScript coverage. These are development data: they choose the configuration to freeze, and are not evidence of generalisation.

7.2.2 Results

All 192 runs completed, none failed. Table 7.4 summarises each arm. Table 7.5 compares each arm with `hybrid_nolearn` on matched (application, seed) pairs.

Table 7.4: Tuning round v3 per arm: 24 runs each on the eight rebuilt development applications, which hold 41 seeded faults (8 of them crash-type).

Arm	Found a fault	RMST	Distinct faults	Crash faults	Unique failures / run	JS coverage
`dev_v3_failfeedback`	21 / 24	**450**	19 / 41	3 / 8	3.79	0.893
`random`	**23** / 24	498	19 / 41	1 / 8	5.38	0.861
`dev_v3_browse`	20 / 24	545	19 / 41	3 / 8	3.12	0.885
`hybrid_nolearn`	21 / 24	560	19 / 41	**4** / 8	2.88	0.896
`v1_dqn_pretrained`	20 / 24	564	19 / 41	2 / 8	4.75	**0.909**
`dev_v3_clickables`	19 / 24	638	17 / 41	3 / 8	2.50	0.891
`dev_v3_curiosity`	20 / 24	650	16 / 41	2 / 8	**5.67**	0.894
`hybrid_bandit`	19 / 24	692	16 / 41	3 / 8	2.83	0.896

Table 7.5: Tuning round v3 against `hybrid_nolearn`: per-application mean difference in T (negative = faster), median over applications with a 95% bootstrap interval. Apps with no difference are not counted as faster or slower.

Arm	Median	95% interval	Apps faster / slower
`dev_v3_browse`	−20	$[-79, 0]$	6 / 1
`dev_v3_failfeedback`	0	$[-379, 0]$	3 / 1
`random`	−18	$[-285, +186]$	5 / 3
`v1_dqn_pretrained`	−45	$[-182, +265]$	5 / 3
`dev_v3_clickables`	0	$[0, +286]$	0 / 3
`dev_v3_curiosity`	+43	$[-248, +549]$	4 / 4
`hybrid_bandit`	+140	$[+112, +416]$	1 / 7

Reading.

- `browse_list` **helps consistently but modestly.** It was faster on six of seven applications and slower on one, and lost no fault.
- **Failure feedback gave the best restricted mean time** (450).
- **Custom clickables did not help.** They were never faster, and verified fewer distinct faults.
- **The bandit was slower than no learner** on seven of eight applications. Its interval excludes zero.
- **Curiosity Q-learning triggered the most unique browser errors** (5.67 per run) but verified fewer seeded faults.
- **Random exploration was strong on these applications.** It found a fault in 23 of 24 runs and triggered many errors, but verified only one crash-type fault.
- **The rebuilt applications are hard.** 15 of the 41 faults were verified by no arm.

7.2.3 Configuration chosen

The two options that did not hurt any measure, `browse_list` and failure feedback, were combined (`dev_v3_combined`) and checked in 24 further runs on the same applications and seeds before freezing. The combination found a fault in 22 of 24 runs, with restricted mean T 465. It verified 22 of the 41 faults, more than any single arm, and 4 of the 8 crash-type faults. Against `hybrid_nolearn` it was faster on six of seven applications (median -52, interval $[-145, 0]$). It was frozen as `hybrid_final`, and the commit was tagged before anyone looked at the external benchmark's applications. Custom clickables and the bandit were not adopted.

7.3 Comparison with WebExplor and WebRLED

7.3.1 Design

The WebRLED paper [1] reports results on WebExplor's six subject applications (Dimeshift, Pagekit, Splittypie, Phoenix, Retroboard and Petclinic), with one-hour runs and five repetitions. WebExplor averaged 55.54% client-side branch coverage and 3.1 failures; WebRLED averaged 64.18% and 4.1. Coverage was measured with Istanbul on the instrumented client code. Failures were console errors, deduplicated by hand.

The comparison was pre-registered when `hybrid_final` was frozen, before anyone looked at the subject applications.

- **Applications.** The same six applications, from the Docker images of the DIG replication material [15], which WebExplor's benchmark reuses. Each image instruments its client JavaScript with Istanbul before the application starts.

- **Arms.** `hybrid_final`; the hybrid with curiosity Q-learning, the RL arm closest to WebExplor and WebRLED; and random.
- **Runs.** One hour of wall-clock time per run, three repetitions (seeds 0–2), and a fresh application instance per run, so $3 \times 6 \times 3 = 54$ runs.
- **Measures.** Branch coverage of the instrumented client code, read from Istanbul's counters after every step and unioned over the run; and unique failures, meaning console errors, page errors and failed requests, deduplicated automatically by kind, page and normalised message.
- **Analysis.** A descriptive comparison with the published averages, with no significance test against numbers from other machines and runs, and a per-application comparison between our arms.

Two decisions were made during set-up and recorded before any run. First, Pagekit and Phoenix start with a sign-in macro using their documented accounts. Second, requests to a development live-reload server and to web analytics, which the images' pages make, are answered with an empty response. Left alone, they would add the same console failure to every run of every arm. Two differences from the published protocol remain and are reported: three repetitions instead of five, and automatic instead of manual deduplication of failures.

To finish in time, the runs were split across two machines, decided and recorded before any result existed. Dimeshift, Pagekit and Splittypie ran on the development laptop; Phoenix, Retroboard and Petclinic ran on four AWS spot instances (2 vCPU, 8 GB). Every run of a given application used the same machine type, so arms are compared on equal hardware within each application. The AWS applications also ran on GitHub Actions as a replication. Which runs count was fixed in advance by completion, never by result. After launch, Phoenix's sign-in was found to complete about a second after the macro ended, so episodes began on a stale page. The macro was fixed, and every earlier Phoenix run was discarded unread.

7.3.2 Results

All 54 counted runs completed, 27 on the laptop and 27 on AWS. The GitHub Actions replication also completed all 27 of its runs. Phoenix runs made before the sign-in fix were discarded on every platform without being read, as recorded in the pre-registration. Table 7.6 gives branch coverage per application.

Table 7.6: External comparison: client-side branch coverage (%), mean [min, max] over three one-hour runs, beside the published means (five runs) of WebExplor and WebRLED [1].

App	Branches	`hybrid_final`	`hybrid_curiosity`	`random`	WebExplor	WebRLED
Dimeshift	1,080	49.2 [47.5, 52.4]	47.8 [46.9, 48.7]	45.0 [44.4, 45.8]	54.2	55.4
Pagekit	555	32.4 [32.1, 32.8]	33.9 [30.8, 39.0]	34.2 [32.1, 35.5]	25.8	39.8
Splittypie	130	35.7 [32.6, 40.8]	42.9 [38.4, 49.1]	37.5 [32.7, 42.4]	36.2	44.6
Phoenix	76	48.7 [48.7, 48.7]	48.7 [48.7, 48.7]	46.9 [43.4, 48.7]	71.8	82.9
Retroboard	171	53.2 [53.2, 53.2]	53.2 [53.2, 53.2]	53.2 [53.2, 53.2]	60.2	77.4
Petclinic	20	46.7 [20.0, 75.0]	41.7 [20.0, 85.0]	25.0 [15.0, 40.0]	85.0	85.0
Average		44.3	44.7	40.3	55.5	64.2

Failures. The pre-registered failure count averaged 21.9 per run for `hybrid_final`, 31.8 for `hybrid_curiosity` and 19.2 for random. These numbers cannot be compared with the papers' 3.1 and 4.1. The pre-registered count includes failed requests as well as console errors. It counts a message once per page. And it includes requests the applications make to other websites: Pagekit's Gravatar images, Splittypie's geo-location service, and external links. The papers counted console errors and merged by hand those that "differ only in parameters". A post-hoc count closer to theirs keeps console errors only, drops messages naming another host, and replaces ids, numbers, quoted values and codes with placeholders before counting distinct messages.

Table 7.7: Post-hoc: distinct console error messages per run (mean [min, max]), the measure closest to the papers' manually deduplicated failures.

App	`hybrid_final`	`hybrid_curiosity`	`random`	WebExplor	WebRLED
Dimeshift	3.0 [1, 4]	2.7 [2, 3]	1.3 [1, 2]	3.0	5.4
Pagekit	2.0 [1, 3]	4.0 [1, 7]	2.7 [2, 3]	6.6	7.4
Splittypie	1.0 [1, 1]	1.0 [1, 1]	1.0 [1, 1]	4.0	5.0
Phoenix	3.0 [3, 3]	3.0 [3, 3]	3.0 [3, 3]	0.0	2.0
Retroboard	0.0 [0, 0]	0.0 [0, 0]	0.0 [0, 0]	1.0	1.0
Petclinic	2.0 [2, 2]	2.7 [2, 4]	0.7 [0, 1]	3.8	3.8
Average	1.8	2.2	1.4	3.1	4.1

Between our arms. Both hybrid arms reached more code than random on average (44.3% and 44.7% against 40.3%). The per-application difference is small and its interval includes zero, for `hybrid_final` (median +0.9 points, $[-1.8, +12.9]$) and for `hybrid_curiosity` (+2.3, $[-0.2, +11.1]$). The curiosity arm triggered more distinct failures than random on every application where they differed (post-hoc count: median +0.7, three applications higher, none lower), consistent with its development-round behaviour. With six applications these comparisons are descriptive.

Replication. On GitHub's runners (4 vCPU, 16 GB), Phoenix and Retroboard gave exactly the same coverage as on AWS for every arm, and the same post-hoc failure counts. Petclinic, which has only 20 branches, varied as much between platforms as between seeds: for `hybrid_final`, 35.0% against 46.7%.

Against the published numbers. The frozen tester does not reach the published averages. It is comparable on Pagekit and Splittypie, below WebExplor and WebRLED on Dimeshift, Phoenix, Retroboard and Petclinic, and below both on average (44.3% against 55.5% and 64.2%; 1.8 failures against 3.1 and 4.1).

Why: the action space (exploratory). On Phoenix and Retroboard, no run of any arm, random included, went beyond the same coverage: 48.7% and 53.2%. Both hybrid arms reached exactly that in every run. When three different exploration strategies, one of them random, stop at the same branch, the limit is what they can act on, not how they choose. Phoenix is a React application whose controls are mostly clickable `div` elements. The frozen tester offers only standard HTML controls, because the custom-clickables option (Section 5.5) was not adopted in tuning. The post-hoc follow-up in Section 7.4 tests this directly. WebRLED learns which elements are actionable with a trained discriminator [1]; on these applications, that part of its design matters more than the choice of policy.

7.4 Post-hoc follow-up: the action space

The external result suggested that the tester's action space, not its policy, limited coverage. To measure this, all three external arms were run again with custom clickables in the action space. That is the only change; tag `v4-clickables` marks it. Every run of this follow-up, with and without clickables, ran on GitHub's runners. For Dimeshift, Pagekit and Splittypie the arms without clickables were run there anew; for the other three, the replication runs serve. This version was defined after the external result. It is a diagnosis and does not replace the pre-registered result.

Table 7.8: Post-hoc follow-up: branch coverage (%, mean of three one-hour runs) without and with custom clickables, all on the same hardware.

App	`hybrid_final`	`hybrid_curiosity`	`random`	WebExplor	WebRLED
Dimeshift	47.5 → 49.3	48.0 → 48.6	45.4 → 45.2	54.2	55.4
Pagekit	33.2 → 33.3	35.1 → 34.5	34.2 → 37.1	25.8	39.8
Splittypie	33.1 → 34.9	45.6 → 43.2	37.5 → 37.7	36.2	44.6
Phoenix	48.7 → **82.0**	48.7 → **83.3**	46.9 → **74.6**	71.8	82.9
Retroboard	53.2 → 53.2	53.2 → 53.2	53.2 → 53.2	60.2	77.4
Petclinic	35.0 → 20.0	85.0 → 61.7	28.3 → 26.7	85.0	85.0
Average, 6 apps	41.8 → 45.5	52.6 → 54.1	40.9 → 45.7	55.5	64.2
Average, without Petclinic	43.1 → 50.6	46.1 → 52.6	43.5 → 49.5	49.6	60.0

Reading.

- **Phoenix.** With clickables, every arm rose by 28 to 35 points. The hybrid arms reached 82–83%, level with WebRLED's 82.9% and above WebExplor's 71.8%. Random reached 74.6%. The Phoenix gap was the action space.
- **Retroboard.** The ceiling (53.2% for every arm and run) did not move. Its cause is not the clickable elements and was not identified.
- **Dimeshift, Pagekit and Splittypie.** Small changes of a few points in either direction. Clickables were not what limited them.
- **Petclinic.** With only 20 branches, one branch is 5 points. Results varied between platforms and seeds as much as between conditions; the three-run means cannot be read as an effect of clickables.
- **Failures.** Distinct console failures did not change (about two per run for every arm).

Against the published numbers. Over all six applications, the arms with clickables averaged 45.5–54.1%, still below WebExplor's 55.5% and WebRLED's 64.2%. Petclinic, where WebExplor and WebRLED both report 85%, weighs heavily on that average. Without Petclinic, a post-hoc view, the arms with clickables averaged 49.5–52.6%. That matches or exceeds WebExplor (49.6%) and remains below WebRLED (60.0%).

What it shows about policy. The gains were similar for the frozen tester, the curiosity arm and random. What the testers could act on mattered more than how they chose among the actions they had.

7.5 Discussion

What the structure contributes. Across every evaluation, the parts of the hybrid tester that are not learned did the measurable work. Remembering states and returning to them exactly, abstracting pages, and applying test-design operators let it verify deep faults that random exploration did not reach: nine faults no random run found on the held-out applications, and the most distinct faults of any configuration in the final tuning round (22 of 41). The verified-fault oracle produced no false positive in any clean-build control.

What learning contributes.

- **Deep RL (v1).** Pretraining did not transfer, and the network did not learn which action helps in which state.
- **Bandit layer.** Thompson sampling over operators was never better than trying untried options systematically, and in the final round it was slower.

- **Curiosity Q-learning.** It reliably triggered more distinct browser errors, on the development applications and on the external benchmark, but verified fewer seeded faults.

This agrees with the observation from Humanoid that a learned model may not be the main reason for a tester's gains [9]. It also agrees with the web-testing study that found coverage only weakly related to failure revelation [7]: the arm that causes the most errors is not the one that verifies the most faults.

How the tester compares with the literature. On WebExplor's own applications and measures, the frozen tester did not reach WebExplor's or WebRLED's published coverage or failure counts. The post-hoc follow-up located a large part of the gap in the action space, not the exploration policy. With custom clickables, Phoenix coverage rose to WebRLED's level for every arm, and over the five applications other than Petclinic the arms matched or exceeded WebExplor's average, though not WebRLED's. Which elements a tester can act on bounds every policy built on it. The comparison also has limits. It used three repetitions instead of five, different machines, automatic instead of manual failure deduplication, and published numbers we could not re-run.

Two measures, two pictures. The project's own evaluations measure verified seeded faults; the literature measures coverage and console errors. The hybrid was designed and tuned for the first, and is strongest there. On the second it is close to random on coverage and fewer failures. A tester for real use needs both: deep, verified functional faults, and broad reach over the interface.

Which arm is best depends on the measure. Table 7.9 gathers, for each measure, the arm that did best and whether it learns.

Table 7.9: The best arm per measure, across all evaluations.

Measure	Best arm	Reinforcement learning?
Distinct verified faults, final tuning round	`hybrid_final` (22 of 41)	no
Distinct verified faults, held-out evaluation	`hybrid_bandit` (19 of 24)	bandit only
Restricted mean time to first fault, held-out	`v1_dqn_pretrained` (595)	yes, deep Q-learning
Client-side coverage, final tuning round	`v1_dqn_pretrained` (0.91)	yes, deep Q-learning
Branch coverage, WebExplor's applications	`hybrid_curiosity` (44.7%)	yes, Q-learning
Distinct failures triggered (all evaluations)	`hybrid_curiosity`	yes, Q-learning

The pattern is consistent. Arms that learn to explore, curiosity Q-learning and the deep Q-network, do best on the literature's measures: breadth of coverage and errors triggered. The structure without a learner does best on verified seeded faults, which require deliberate test actions rather than novelty. No learning layer added on top of the structure verified more faults than the structure alone. The like-for-like RL comparison with WebExplor and WebRLED,

both RL testers, is therefore `hybrid_curiosity`. It reached 44.7% branch coverage; with custom clickables it reached 54.1% over all six applications, level with WebExplor (55.5%) and below WebRLED (64.2%).

Limits of the evidence.

- **Scale.** Every evaluation used six to eight applications, which leaves wide intervals; all four held-out comparisons were inconclusive.
- **Same developer.** The development and validation applications were built within the project. The external applications were not, and there the tester did worse.
- **Seeded faults.** Seeded faults stand in for real ones only partially.
- **Post-hoc analyses.** The failure measure comparable with the papers, the clickables follow-up and the view without Petclinic were defined after the results and are reported as such.
- **Phoenix cold start.** On a freshly started container, Phoenix's first sign-in redirect can complete after the macro's wait, so a run's first episode may begin on the sign-in page. This affects at most one episode per run, equally for every arm.

Chapter 8

Conclusion and Scope for Further Research

8.1 Conclusion

This thesis asked whether reinforcement learning can make an automated functional tester for web applications find verified faults sooner, and under what design. The answer has three parts.

First, a deep RL tester (v1) learned useful behaviour inside an application, but not in a way that transferred or used context. Pretraining on five applications did not speed up discovery on a sixth, and the network ranked actions without regard to whether they helped in the current state. A small local LLM did not help, either as a progress reward or as a page labeller; neither generalised beyond the data it was tuned on.

Second, a hybrid tester built around structure rather than learning (page abstraction, an archive with exact return, test-design operators) verified deep faults that random exploration missed, with no false positives in any control run. On held-out applications its arms verified the most distinct faults, though no pre-registered comparison of speed reached a verdict. Across every evaluation, no learning layer improved on the same structure without learning.

Third, on WebExplor's own benchmark, measured as the published work measures, the frozen tester did not reach WebExplor's or WebRLED's coverage or failure counts. A post-hoc follow-up located much of the gap in the set of elements the tester can act on. With custom clickables, Phoenix coverage rose from 49% to WebRLED's 83% for every arm. Excluding the 20-branch Petclinic, the arms matched or exceeded WebExplor's average, though not WebRLED's.

The evidence therefore supports a modest claim. For finding verified functional faults, the structure of the tester matters more than the learning in it. For broad reach over an interface, what a tester can perceive and act on bounds any policy. Learning, as used here and in the literature it builds on, was not what made the difference.

8.2 Limitations

- Each evaluation used six to eight applications, so most comparisons are inconclusive and the claims are descriptive.
- The development and validation applications and their seeded faults were built within the project.
- Seeded faults are an incomplete proxy for real faults, and the oracle verifies only what was seeded; the specification-free invariants raised false alarms on unsaved state.
- The external comparison used three repetitions instead of five, different machines, and automatic failure deduplication, against published numbers that could not be re-run.
- Repository intake requires a Dockerfile.

8.3 Scope for further research

- **Action space.** Make the custom-clickables scan the default and evaluate it under pre-registration, or learn which elements are actionable, as WebRLED does; and find what bounds Retroboard.
- **Two deviations in one walk.** Operators that combine two deviations in one walk, for the faults no single deviation reaches.
- **Learning across applications.** At the scale the literature suggests (tens of applications), with page semantics in the state.
- **Specification-free oracles** that distinguish saved from unsaved state.
- **LLMs off the decision path.** Use them offline, to propose test operators or assertions for an application.

Appendix A

Agent Configurations

A.1 Arms

Table A.1: Every arm used in the evaluations, by generation.

Arm	Configuration
Baselines	
`random`	uniform choice among the page's valid actions
`search_A0`	fixed hand-weighted action priority; no learning
`search_S`	fixed linear score over the action features; no learning
v1: action-conditioned DQN (Chapter 4)	
`dqn_scratch`	learns on the target only; LLM reward on
`dqn_scratch_nollm`	as above, LLM reward off
`dqn_pretrained`	pretrained on the other five applications with the LLM reward; residual head adapts
`dqn_pretrained_nollm`	pretrained without the LLM reward; residual head adapts (`v1_dqn_pretrained` in v2 evaluations)
`dqn_zeroshot`	pretrained network frozen; no adaptation
`bandit_scratch`, `bandit_pretrained`	the same network with $\gamma = 0$
v2: hybrid tester (Chapter 5); all with depth preference $\beta_d = 2$	
`hybrid_nolearn`	untried (operator, target) pairs first, at random
`hybrid_bandit`	Thompson-sampling bandit over semantic keys
`dev_bandit_novelty`, `dev_bandit_surprise`	bandit rewarded only for new pages / for unexpected responses
`dev_pairwise_bandit`, `dev_pairwise_nolearn`	pairwise operator and technique-first choice
`dev_hybrid_curiosity`	curiosity Q-learning chooses the free exploration
`dev_curiosity_rl`	standalone curiosity Q-learning tester (no operators)
`dev_v3_browse`	with the `browse_list` operator
`dev_v3_clickables`	with custom clickables in the action registry
`dev_v3_curiosity`	curiosity Q-learning for free exploration (rebuilt suite)
`dev_v3_failfeedback`	archive weight for cells where errors followed actions

Table A.2: Hyperparameters.

Component	Values
Run	budget 2,000 interactions; episodes of at most 40 actions
v1 network	state encoder 64, action encoder 96, scorer 128 units; dueling
v1 learning	Double DQN, 3-step returns, $\gamma = 0.95$, learning rate 3×10^{-4}, batch 64, buffer 30,000, target update 200, PER $\alpha = 0.6$, $\beta_0 = 0.4$
v1 exploration	guide probability 0.75; target ϵ 0.50 $\rightarrow$ 0.05 over half the budget; pretraining 1,500 steps per application
v1 reward	LLM weight $\lambda = 1$ (LLM arms), outcome bonus 3, reveal cap 1.5, clip ± 6
Hybrid archive	weight of Section 5.2; depth preference $\beta_d = 2$
Bandit	Beta posteriors; back-off weight 0.3
Curiosity Q-learning	$\alpha = 0.5$, $\gamma = 0.9$, $\epsilon = 0.1$, episodic bonus 0.5, optimistic initialisation $R_{\max}/(1-\gamma)$

A.2 Held-out evaluation per fault

Table A.3: Held-out evaluation: runs (of 5) that verified each fault.

Fault	R	V1	HN	HB	Fault	R	V1	HN	HB
DUTY-1	3	3	0	3	QUIZ-1	5	5	5	5
DUTY-2	4	3	4	4	QUIZ-2	0	2	1	2
DUTY-3	0	1	1	0	QUIZ-3	0	1	2	0
FOST-1	5	3	4	5	SIDE-1	0	0	1	1
FOST-2	0	0	0	0	SIDE-2	1	1	2	2
FOST-3	0	2	1	1	SIDE-3	1	0	4	3
LARD-1	1	0	3	4	SKY-1	0	3	0	1
LARD-2	5	5	5	5	SKY-2	3	1	0	1
LARD-3	1	0	2	1	SKY-3	5	5	0	2
MARK-1	0	0	2	2	TALLY-1	5	5	5	5
MARK-2	0	1	2	1	TALLY-2	0	2	1	0
MARK-3	1	4	1	1	TALLY-3	0	0	0	0

R = `random`, V1 = `v1_dqn_pretrained`, HN = `hybrid_nolearn`, HB = `hybrid_bandit`.

References

[1] Z. Gu, C. Liu, G. Wu, Y. Zhang, C. Yang, Z. Liang, W. Chen, and J. Wei, "Deep reinforcement learning for automated web GUI testing," 2025. arXiv preprint; introduces WebRLED. A journal article on WebRLED by an overlapping author list is registered at doi:10.1145/3843235 (ACM TOSEM, 2026).

[2] Y. Zheng, Y. Liu, X. Xie, Y. Liu, L. Ma, J. Hao, and Y. Liu, "Automatic web testing using curiosity-driven reinforcement learning," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 423–435, 2021.

[3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[4] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[5] ISO/IEC, "ISO/IEC 25010:2011 systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models." International Organization for Standardization, 2011.

[6] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2018.

[7] C. Liu, W. Yang, Y. Zhang, and T. Xie, "Understanding automated web GUI testing: An empirical study across exploration strategies and state abstractions," 2026. arXiv preprint.

[8] Y. Zhao, B. Harrison, and T. Yu, "DinoDroid: Testing Android apps using deep Q-networks," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 5, 2024.

[9] Y. Li, Z. Yang, Y. Guo, and X. Chen, "Humanoid: A deep learning-based approach to automated black-box Android app testing," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1070–1073, 2019.

[10] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level

control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[11] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, 2016.

[12] J. He, J. Chen, X. He, J. Gao, L. Li, L. Deng, and M. Ostendorf, "Deep reinforcement learning with a natural language action space," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016. Introduces the deep reinforcement relevance network (DRRN).

[13] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune, "First return, then explore," *Nature*, vol. 590, no. 7847, pp. 580–586, 2021. The Go-Explore family of algorithms.

[14] D. J. Russo, B. V. Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on Thompson sampling," *Foundations and Trends in Machine Learning*, vol. 11, no. 1, pp. 1–96, 2018.

[15] M. Biagiola, A. Stocco, F. Ricca, and P. Tonella, "Diversity-based web test generation," in *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019.